

COS AT-3 Flowchart & Coding

Name: H. Harshwardhan

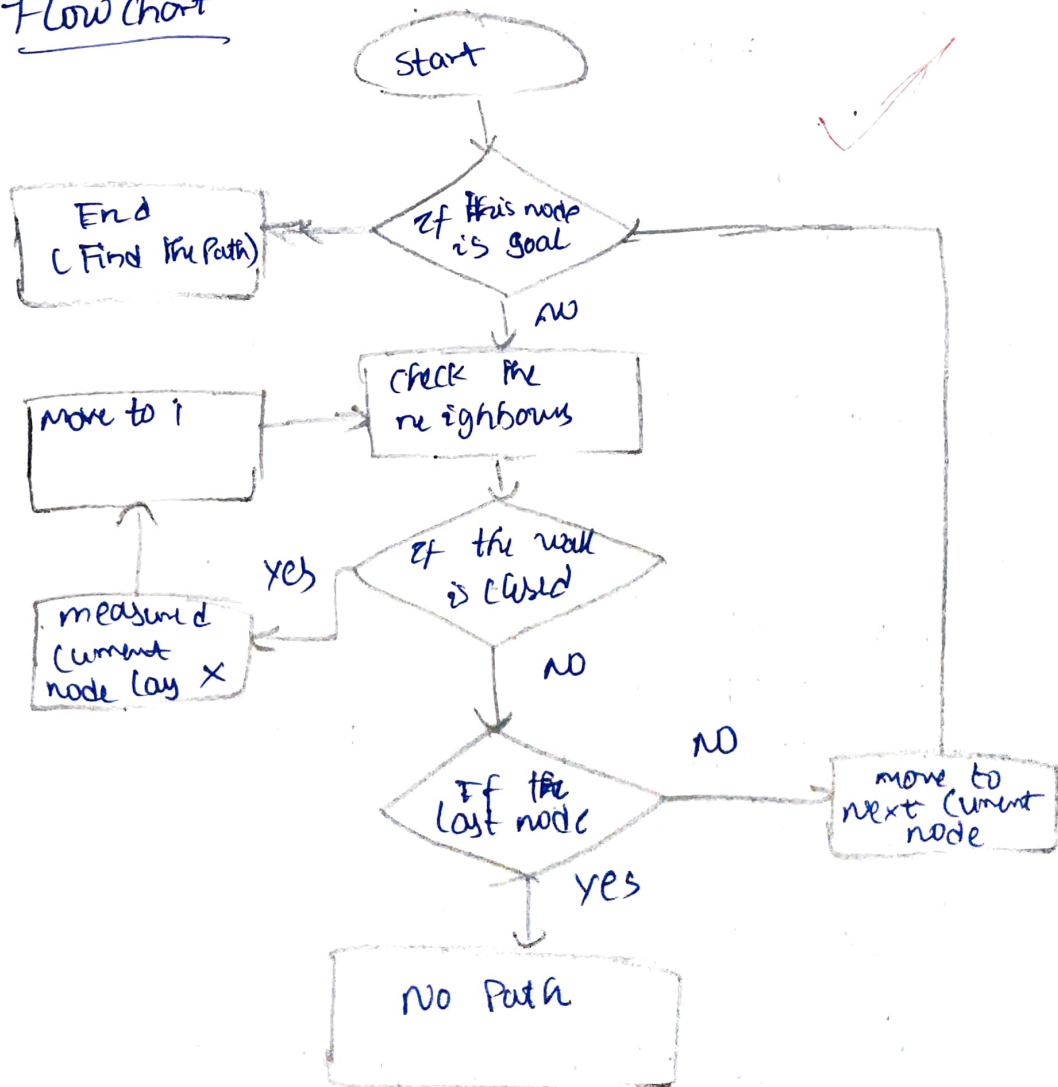
Slot: D

Regno: 192511171

Sub: CSA0315 Data structures

- 1) Convert a flowchart for Depth first search using a Stack into a C coding
- 2) A flowchart implements a shortest path algorithm, Dijkstra algorithm and convert it into C code
- 3) A flowchart perform prim's algorithm, convert the flowchart into a recursive C programming.

1) Flowchart



Coding:

```
#include <stdio.h>
```

```
#define MAX 10
```

```
int graph [MAX][MAX];
```

```
int visited [MAX];
```

```
int n;
```

```
void DFS (int vertex)
```

```
{  
    int i;
```

```
    visited [vertex] = 1;
```

```
    printf ("%d ", vertex);
```

```
    for (i=0; i<n; i++)
```

```
    {
```

```
        if (graph [vertex][i] == 1 && visited [i] == 0)
```

```
        {
```

```
            DFS (i);
```

```
        }
```

```
    }
```

```
int main ()
```

```
{
```

```
    int i, j, start;
```

```
    printf ("Enter number of vertices: ");
```

```
    scanf ("%d", &n);
```

```
    printf ("Enter adjacency matrix:\n");
```

```
    for (i=0; i<n; i++)
```

```
    {  
        for (j=0; j<n; j++)
```

```
        {
```

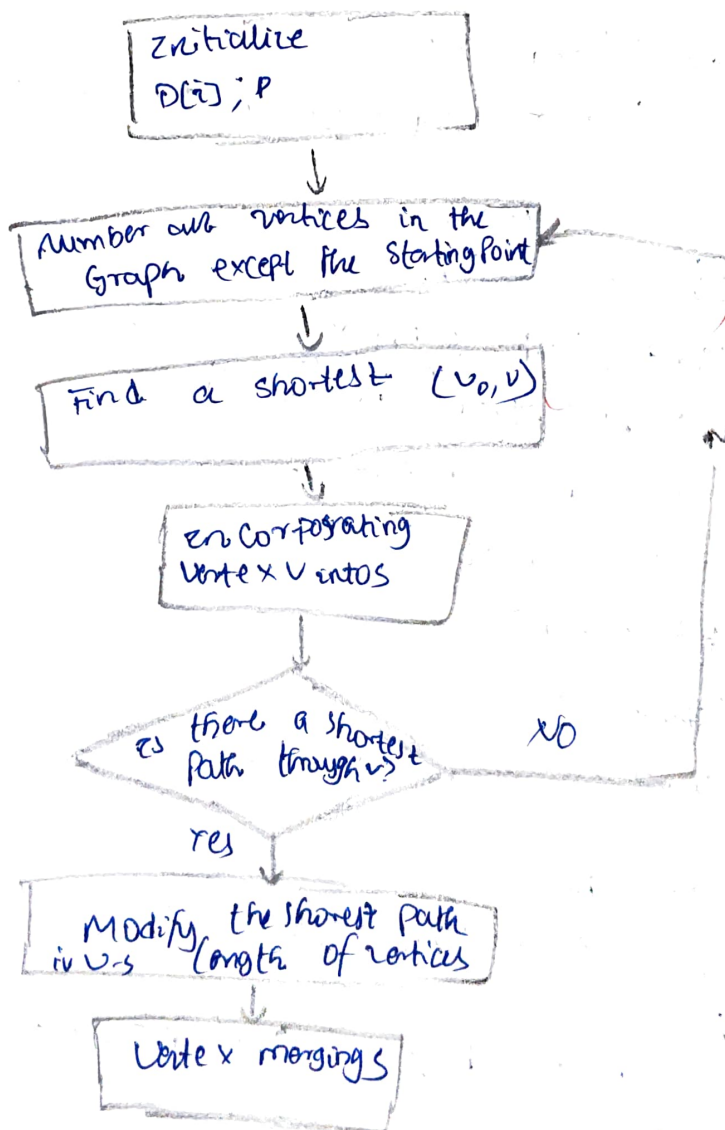
```
            scanf ("%d", &graph [i][j]);
```

```

y
y
printf ("Enter starting vertex:");
scanf ("%d", &start);
printf ("DFS Traversal:");
DFS (start);
return 0;
y

```

2) Flow chart : Shortest path algorithm
Dijkstra algorithm



Coding.

```
#include <stdio.h>
```

```
#define MAX 10
```

```
#define INF 999
```

```
int main()
```

```
{
```

```
    int graph [MAX][MAX], dist [MAX];
```

```
    int visited [MAX] = {0};
```

```
    int n, start, i, j, count;
```

```
    int min, next;
```

```
    printf ("Enter the number of vertices: ");
```

```
    scanf ("%d", &n);
```

```
    printf ("Enter adjacency matrix: \n");
```

```
    for (i=0; i<n; i++)
```

```
        for (j=0; j<n; j++)
```

```
            scanf ("%d", &start);
```

```
    printf ("Enter starting vertex: ");
```

```
    scanf ("%d", &start);
```

```
    for (i=0; i<n; i++)
```

```
        dist [i] = INF;
```

```
    dist [start] = 0;
```

```
    for (count = 0; count < n; count++)
```

```
    {
```

```
        min = INF;
```

```
        for (i=0; i<n; i++)
```

```
        {
```

```
            if (visited [i] == 0 && dist [i] < min)
```

```
        }
```

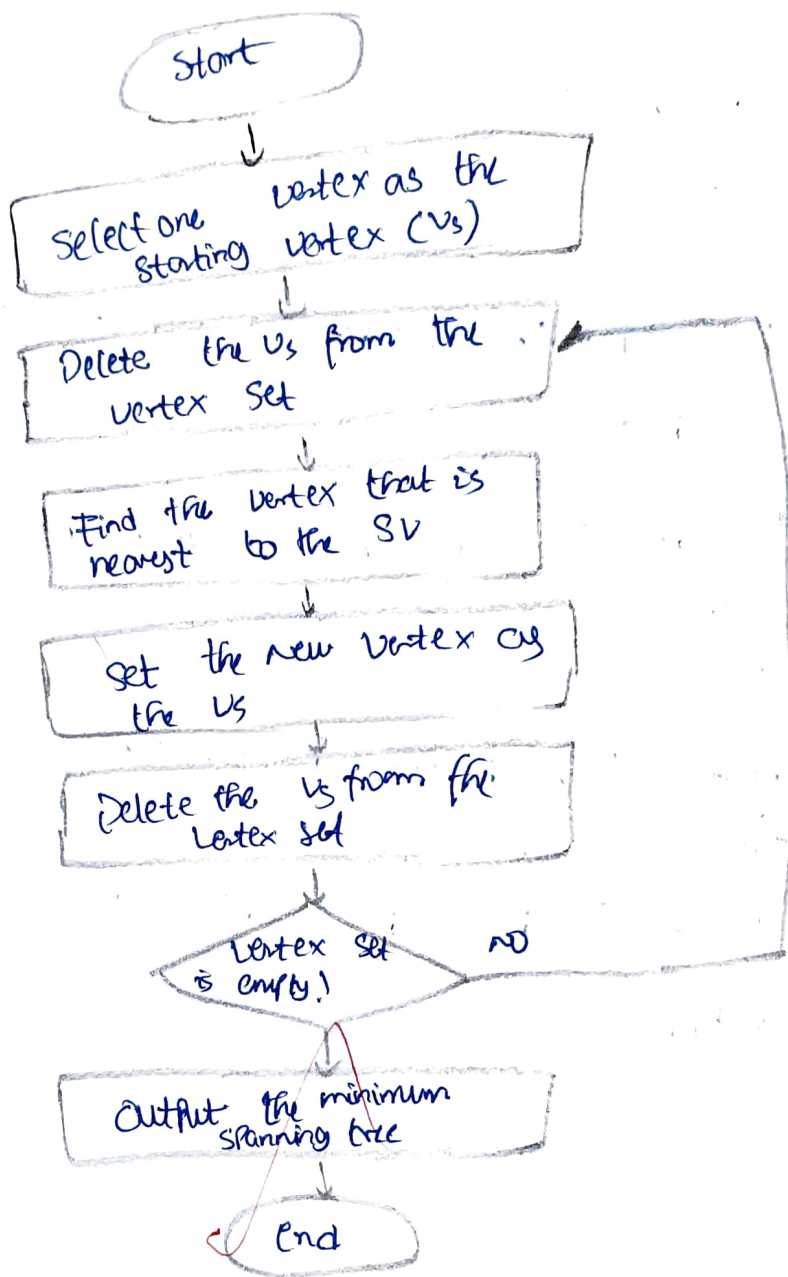
(2)

```

d
    min = dist[i];
    next = i;
}
y
visited [next] = 1;
for (i = 0; i < n; i++)
{
    if (graph [next][i] != 0 && dist [next] + graph [next][i] <
        dist[i])
    {
        dist[i] = dist [next] + graph [next][i];
    }
}
y
y
printf ("n Shortest distances : \n");
for (i = 0; i < n; i++)
    printf ("%d -> %d = %d \n", start, i, dist[i]);
return 0;
y

```

3) flowchart : Prim's algorithm



Coding:

```
#include <stdio.h>
```

```
#define MAX 10
```

```
#define INF 999
```

```
int main()
```

```
{
```

```
    int g[MAX][MAX], v[MAX] = {0};
```

```
    int n, i, j, count = 0, min, a, b, cost = 0;
```

```
    scanf("%d", &n);
```

```
    for (i = 0; i < n; i++)
```

```
        for (j = 0; j < n; j++)
```

```
            scanf("%d", &g[i][j]);
```

```
    v[0] = 1;
```

```
    while (count < n-1)
```

```
    {
```

```
        min = INF;
```

```
        for (i = 0; i < n; i++)
```

```
            if (v[i])
```

```
                for (j = 0; j < n; j++)
```

```
                    if (!v[j] && g[i][j] < min)
```

```
                        min = g[a=i][b=j];
```

```
    printf("%d %d = %d\n", a, b, min);
```

```
    v[b] = 1;
```

```
    cost += min;
```

```
    count++;
```

```
}
```

```
    printf("Minimum cost = %d", cost);
```

```
    return 0;
```

```
}
```